重要模块解释

dp_top.v

一、完整代码

```verilog
module dp_top(
    clk, rst,
    RegDst, RegWr, MemWr,
    ExtOp, nPC_sel, ALUctr,
    MemtoReg, ALUSrc, j_sel,
    instruction_value
);

// 输入端口：时钟、复位、所有控制信号
input clk, rst;
input [1:0] ExtOp, ALUctr;
input ALUSrc, MemWr, MemtoReg, RegWr, j_sel, RegDst, nPC_sel;

// 输出端口：当前取出的指令（供外部查看/调试）
output [31:0] instruction_value;

// 内部连线
wire [31:0] instruction;
wire zero;
wire [31:0] busA, busB, busW, Mux2_out, Mux3_out, imm32, ALU_out, Addr,
data_in, dm_out;
wire [4:0] rw;
wire [4:0] rd;
wire [4:0] rt;
wire [4:0] rs;
wire [15:0] imm16;
```

```
// 把内部指令引到输出，方便观察
assign instruction_value = instruction;

// 从指令中拆分出：rs、rt、rd、16 位立即数
assign rs = instruction[25:21];
assign rt = instruction[20:16];
assign rd = instruction[15:11];
assign imm16 = instruction[15:0];

// 1. 取指单元：PC 更新、分支/跳转、从指令内存取指令
IFU ifu(
    .nPC_sel(nPC_sel),
    .zero(zero),
    .instruction(instruction),
    .clk(clk),
    .rst(rst),
    .j_sel(j_sel)
);

// 2. 写回地址选择：R 型写 rd，I 型写 rt
MUX1 mux1(
    .RegDst(RegDst),
    .rd(rd),
    .rt(rt),
    .out(rw)
);

// 3. 通用寄存器堆：读 rs/rt、写 rw
GPR gpr(
    .rw(rw),
    .ra(rs),
    .rb(rt),
    .regWr(RegWr),
    .busW(Mux3_out),
    .clk(clk),
    .busA(busA),
    .busB(busB),
```

```
    .data_in(data_in),
    .rst(rst)
);

// 4. 立即数扩展：16 位 → 32 位（零扩展/符号扩展/高位扩展）
EXT ext(
    .imm16(imm16),
    .out32(imm32),
    .ExtOp(ExtOp)
);

// 5. ALU 输入选择：来自寄存器 busB 或 扩展立即数 imm32
MUX2 mux2(
    .ALUSrc(ALUSrc),
    .busB(busB),
    .ext_out(imm32),
    .out(Mux2_out)
);

// 6. ALU：做加减、或运算；输出结果、零标志、地址
ALU alu(
    .data1(busA),
    .data2(Mux2_out),
    .zero(zero),
    .Addr(Addr),
    .ALUctr(ALUctr),
    .out(ALU_out)
);

// 7. 数据存储器：写内存（sw）、读内存（lw）
DM data_memory(
    .data_in(data_in),
    .clk(clk),
    .WrEn(MemWr),
    .Addr(Addr),
    .out(dm_out),
    .rst(rst)
```

```
);

// 8. 写回数据选择：来自 ALU 结果 或 内存读出数据
MUX3 mux3(
    .MemtoReg(MemtoReg),
    .ALU_out(ALU_out),
    .DM_out(dm_out),
    .out(Mux3_out)
);

endmodule
```

逐段详细解释

1. 端口定义

```verilog
input clk, rst;
input [1:0] ExtOp, ALUctr;
input ALUSrc, MemWr, MemtoReg, RegWr, j_sel, RegDst, nPC_sel;
output [31:0] instruction_value;
```

- `clk/rst`：时钟、复位。
- 所有 `input`：**控制器发来的控制命令**（选哪路、写不写、怎么扩展…）。

`instruction_value`：输出当前指令，方便看。

2. 内部连线（wire）

都是**中间数据 / 信号**，模块之间传递用：

- `instruction`：当前 32 位指令。
- `busA/busB`：从寄存器读出的两个数。
- `imm32`：扩展后的 32 位立即数。
- `ALU_out`：ALU 结果。

1. `dm_out`：从内存读出的数据。
2. `rw`：要写回的寄存器编号。

3. 拆分指令字段

```verilog
assign rs = instruction[25:21];
assign rt = instruction[20:16];
assign rd = instruction[15:11];
assign imm16 = instruction[15:0];
```

把 32 位指令拆开，送给后面模块：

- `rs/rt`：源寄存器（读哪两个）。
- `rd`：目的寄存器（R 型）。
- `imm16`：16 位立即数（I 型）。

IF_ID.v

1. 完整代码

```verilog
module IF_ID(
    instruction, next_PC, clk, instruction_out, reset, PC_out
);

input [31:0] instruction, next_PC;
input clk, reset;

output reg [31:0] instruction_out, PC_out;

always@(posedge clk, posedge reset)
```

```
begin
    if(reset)begin
        instruction_out <= 0;
        PC_out <= 0;
    end
    else begin
        instruction_out <= instruction;
        PC_out <= next_PC;
    end
end

endmodule
```

2. 逐行逐段详细解释

① 模块定义

```verilog
module IF_ID(...);
```

- 模块名：**IF_ID**
- **功能**：MIPS 5 级流水线中，**IF 取指阶段 → ID 译码阶段** 的**流水线寄存器**。
- **作用**：把取指阶段拿到的指令和地址，**锁存一拍**，传给译码阶段。

---

② 输入端口

```verilog
input [31:0] instruction, next_PC;
input clk, reset;
```

- `instruction[31:0]`：**从指令存储器取出来的 32 位指令**
- `next_PC[31:0]`：**下一条指令的地址（PC+4）**
- `clk`：时钟信号（上升沿有效）

- `reset`：复位信号（高电平复位）

---

③ 输出端口

```verilog
output reg [31:0] instruction_out, PC_out;
```

- `instruction_out[31:0]`：**送给 ID 阶段的指令**
- `PC_out[31:0]`：**送给 ID 阶段的 PC 地址**
- 都是 `reg` 型：因为要在 `always` 里赋值

---

④ 核心逻辑：流水线锁存 + 复位

```verilog
always@(posedge clk, posedge reset)
begin
    if(reset)begin        // 复位：清空指令和地址
        instruction_out <= 0;
        PC_out <= 0;
    end
    else begin            // 时钟上升沿：锁存输入，传给下一级
        instruction_out <= instruction;
        PC_out <= next_PC;
    end
end
```

工作流程：

2. **复位有效时（reset=1）**

输出全部清 0，流水线清空。

3. **时钟上升沿到来时（clk 上升沿）**
    - 把取到的指令 `instruction` → 锁存到 `instruction_out`

- 把下一条地址 `next_PC` → 锁存到 `PC_out`

ID_EX .v

1. 完整代码

```verilog
module ID_EX(RegWriteD, MemtoRegD, MemWriteD, nPC_sel_D, ALUControlD,
        ALUsrcD, RegDstD, RD1_D, RD2_D, RtD, RdD, ext_imm32D, next_PC,
        RegWriteE, MemtoRegE, MemWriteE, nPC_sel_E, ALUControlE,
        ALUsrcE, RegDstE, RD1_E, RD2_E, RtE, RdE, ext_imm32E,
            PC_out, clk, data_in_D, data_in_E,
        j_value_D, j_value_E, j_sel_E, j_sel_D);

// 输入端口
input RegWriteD, MemtoRegD, MemWriteD, ALUsrcD, RegDstD, clk,
nPC_sel_D, j_sel_D;
input [1:0] ALUControlD;
input [31:0] RD1_D, RD2_D, data_in_D;
input [4:0] RtD, RdD;
input [31:0] ext_imm32D;
input [31:0] next_PC;
input [25:0] j_value_D;

// 输出端口
output reg RegWriteE, MemtoRegE, MemWriteE, ALUsrcE, RegDstE,
nPC_sel_E, j_sel_E;
output reg[1:0] ALUControlE;
output reg[31:0] RD1_E, RD2_E, data_in_E;
output reg[4:0] RtE, RdE;
output reg[31:0] ext_imm32E;
output reg[31:0] PC_out;
output reg[25:0] j_value_E;

// 时序逻辑：时钟上升沿锁存所有信号
```

```
always@(posedge clk)
begin
    RegWriteE <= RegWriteD;
    MemtoRegE <= MemtoRegD;
    MemWriteE <= MemWriteD;
    ALUsrcE <= ALUsrcD;
    RegDstE <= RegDstD;
    nPC_sel_E <= nPC_sel_D;
    ALUControlE <= ALUControlD;
    RD1_E <= RD1_D;
    RD2_E <= RD2_D;
    data_in_E <= data_in_D;
    RtE <= RtD;
    RdE <= RdD;
    ext_imm32E <= ext_imm32D;
    PC_out <= next_PC;
    j_sel_E <= j_sel_D;
    j_value_E <= j_value_D;
end

endmodule
```

2. 逐段超通俗解释

模块一句话总结

ID_EX = 流水线第二级寄存器
作用：**把 ID 译码阶段的所有信号，锁存一拍 → 传给 EX 执行阶段**

---

① 模块名与位置

```verilog
module ID_EX(...);
```

- **ID = Instruction Decode（译码阶段）**
- **EX = Execute（执行阶段）**
- 位置：**译码** → **执行**之间

---

② 输入（来自 ID 阶段）

所有信号末尾带 **D** = Decode 译码阶段

- `RegWriteD`：要不要写寄存器
- `MemtoRegD`：数据来自内存还是 ALU
- `MemWriteD`：要不要写内存
- `ALUControlD`：ALU 做加减或运算
- `ALUsrcD`：ALU 第二个数来自寄存器还是立即数
- `RD1_D`、`RD2_D`：寄存器读出的两个数
- `ext_imm32D`：扩展后的立即数

3. `next_PC`：指令地址
4. `j_sel_D`、`j_value_D`：跳转相关信号

---

③ 输出（送往 EX 阶段）

所有信号末尾带 **E** = Execute 执行阶段

- 输出和输入**几乎一模一样**
- 只是把 D 改成 E
- 全部是 `reg` 型，因为要在时钟里赋值

---

④ 核心逻辑（最重要）

```verilog
always@(posedge clk)
```

```
begin
     RegWriteE <= RegWriteD;
     MemtoRegE <= MemtoRegD;
     MemWriteE <= MemWriteD;
     ALUsrcE <= ALUsrcD;
     ...所有信号全部赋值一遍...
end
```

一句话功能：

时钟上升沿一瞬间，把 ID 阶段的所有数据、控制信号，原封不动“打一拍”传给 EX 阶段

EX-MEM.v

完整代码

```verilog
module EX_MEM(RegWriteE, MemtoRegE, MemWriteE, BranchE, clk,
         ALUOutE, WriteDataE, WriteRegE, ext_out_E, zeroE,
         RegWriteM, MemtoRegM, MemWriteM, BranchM, zeroM,
         ALUOutM, WriteDataM, WriteRegM, ext_out_M, next_PC_E,
             next_PC_M, j_sel_E, j_sel_M);

// 输入端口
input RegWriteE, MemtoRegE, MemWriteE, clk, zeroE, BranchE, j_sel_E;
input [4:0] WriteRegE;
input [31:0] ALUOutE, WriteDataE, ext_out_E, next_PC_E;

// 输出端口
output reg RegWriteM, MemtoRegM, MemWriteM, zeroM, BranchM, j_sel_M;
output reg [4:0] WriteRegM;
output reg [31:0] ALUOutM, WriteDataM, ext_out_M, next_PC_M;
```

```
// 时钟上升沿锁存所有信号，传递到下一级
always@(posedge clk)
begin
    RegWriteM   <= RegWriteE;
    MemtoRegM   <= MemtoRegE;
    MemWriteM   <= MemWriteE;
    zeroM       <= zeroE;
    BranchM     <= BranchE;
    WriteRegM   <= WriteRegE;
    ALUOutM     <= ALUOutE;
    WriteDataM  <= WriteDataE;
    ext_out_M   <= ext_out_E;
    j_sel_M     <= j_sel_E;
    next_PC_M   <= next_PC_E;
end

endmodule
```

逐行逐句解释

1. 模块是干什么的？

EX_MEM 是流水线寄存器

位置：

EX 执行阶段 → MEM 访存阶段

作用：

把 EX 阶段算好的所有结果，锁存一下，传给 MEM 阶段继续执行。

---

2. 输入信号（从 EX 来的）

所有带 **E** = EX 执行阶段

- `RegWriteE`：要不要写回寄存器
- `MemtoRegE`：数据来自内存还是 ALU

- `MemWriteE`：是否写内存
- `BranchE`：是否是分支指令
- `zeroE`：ALU 结果是否为 0（用于分支判断）
- `ALUOutE`：ALU 运算结果
- `WriteDataE`：要写入内存的数据
- `WriteRegE`：要写回的寄存器编号
- `next_PC_E`：指令地址
- `clk`：时钟

---

3. 输出信号（送往 MEM 阶段）

所有带 **M** = MEM 访存阶段

输出和输入**完全一样**，只是把后缀 **E → M**

---

4. 核心代码（最关键）

```verilog
always@(posedge clk)
begin
    RegWriteM   <= RegWriteE;
    MemtoRegM   <= MemtoRegE;
    MemWriteM   <= MemWriteE;
    zeroM       <= zeroE;
    BranchM     <= BranchE;
    WriteRegM   <= WriteRegE;
    ALUOutM     <= ALUOutE;
    WriteDataM  <= WriteDataE;
    ext_out_M   <= ext_out_E;
    j_sel_M     <= j_sel_E;
    next_PC_M   <= next_PC_E;
end
```

一句话解释：

当时钟上升沿到来时，把 EX 阶段的所有信号，原封不动传给 MEM 阶段！

MEM_WB.v

1. 完整代码

```verilog
module MEM_WB(
    RegWriteM, MemtoRegM, ALUOutM, ReadDataM, WriteRegM,
    RegWriteW, MemtoRegW, ALUOutW, ReadDataW, WriteRegW, clk
);

// 输入端口（来自 MEM 阶段）
input RegWriteM, MemtoRegM, clk;
input[31:0] ALUOutM, ReadDataM;
input[4:0] WriteRegM;

// 输出端口（送往 WB 阶段）
output reg RegWriteW, MemtoRegW;
output reg [31:0] ALUOutW, ReadDataW;
output reg [4:0] WriteRegW;

// 时钟上升沿锁存，传递信号
always@(posedge clk)
begin
    RegWriteW  <= RegWriteM;
    MemtoRegW  <= MemtoRegM;
    ALUOutW    <= ALUOutM;
    ReadDataW  <= ReadDataM;
    WriteRegW  <= WriteRegM;
end
```

```
endmodule
```

2. 逐行逐句超通俗解释

🔥 一句话核心总结

MEM_WB = 流水线第 4 级寄存器
作用：把 MEM 访存阶段的结果 → 锁存一拍 → 传给 WB 写回阶段

---

1. 模块名是什么意思？

- **MEM = Memory = 访存阶段**（读 / 写内存）
- **WB = Write Back = 写回阶段**（把结果写回寄存器）
- **MEM_WB = 访存 → 写回 之间的流水线寄存器**

---

2. 输入信号（来自 MEM 阶段）

所有带 **M** = 访存阶段来的信号

- `RegWriteM`：是否需要写回寄存器
- `MemtoRegM`：选择写回的数据来源（内存 or ALU）
- `ALUOutM`：ALU 运算结果
- `ReadDataM`：从数据内存读出来的数据
- `WriteRegM`：要写回的寄存器编号（如 1,2 ...）
- `clk`：时钟

MEM_WB.v

1. 完整代码

```verilog
module MEM_WB(
    RegWriteM, MemtoRegM, ALUOutM, ReadDataM, WriteRegM,
    RegWriteW, MemtoRegW, ALUOutW, ReadDataW, WriteRegW, clk
);

// 输入端口（来自 MEM 阶段）
input RegWriteM, MemtoRegM, clk;
input[31:0] ALUOutM, ReadDataM;
input[4:0] WriteRegM;

// 输出端口（送往 WB 阶段）
output reg RegWriteW, MemtoRegW;
output reg [31:0] ALUOutW, ReadDataW;
output reg [4:0] WriteRegW;

// 时钟上升沿锁存，传递信号
always@(posedge clk)
begin
    RegWriteW  <= RegWriteM;
    MemtoRegW  <= MemtoRegM;
    ALUOutW    <= ALUOutM;
    ReadDataW  <= ReadDataM;
    WriteRegW  <= WriteRegM;
end

endmodule
```

2. 逐行逐句超通俗解释

🔥 一句话核心总结

MEM_WB = 流水线第 4 级寄存器
作用：把 MEM 访存阶段的结果 → 锁存一拍 → 传给 WB 写回阶段

---

1. 模块名是什么意思？

- **MEM = Memory = 访存阶段**（读 / 写内存）
- **WB = Write Back = 写回阶段**（把结果写回寄存器）
- **MEM_WB = 访存 → 写回 之间的流水线寄存器**

---

2. 输入信号（来自 MEM 阶段）

所有带 **M** = 访存阶段来的信号

- `RegWriteM`：是否需要写回寄存器
- `MemtoRegM`：选择写回的数据来源（内存 or ALU）
- `ALUOutM`：ALU 运算结果
- `ReadDataM`：从数据内存读出来的数据
- `WriteRegM`：要写回的寄存器编号（如 1,2 ...）
- `clk`：时钟

IFU.v

1. 完整代码

```verilog
module IFU(nPC_sel, zero, instruction, clk, rst, j_sel);
input nPC_sel, zero, clk, rst;
input j_sel;
output[31:0] instruction;

reg[7:0] Instrution_Memory[1023:0];
reg[31:0] PC_value;

wire[31:0] PC_new;
wire[31:0] PC_ext;
wire[15:0] imm16;
```

```
wire[31:0] ins0;
wire[31:0] ins1;
wire[25:0] j_value;
wire[31:0] temp_1;
wire[31:0] temp_2;

assign j_value = instruction[25:0];
assign imm16 = instruction[15:0];

assign temp_1 = {{16{imm16[15]}},imm16};
assign temp_2 = {PC_value[31:28], j_value[25:0], 2'b0};

// declare the reset operation of PC value
always@(posedge clk, posedge rst)
begin
if(rst)
// reset PC to 0x0000 3000
PC_value <= 32'h0000_3000;
else if(j_sel==1'b1)
PC_value <= PC_ext;
else
PC_value <= PC_new;
end



// considering the jump condition when extending the instruction
assign PC_ext = j_sel ? temp_2:temp_1 << 2;

// obtain the address of the next instruction in two cases:
// one is the normal senario
assign ins0 = PC_value + 4;
// the other is the beq scenario
assign ins1 = ins0 + PC_ext;
// assign the value with PC
assign PC_new = (nPC_sel && zero) ? ins1 : ins0;

// concatenate the final value of instruction, which is composed of
```

```
foure byte.
// note that PC_value is a 32-bit value; therfore, it is necesary to
convert it to 10-bit.
assign instruction = {Instrution_Memory[PC_value[9:0]],
Instrution_Memory[PC_value[9:0]+1],
Instrution_Memory[PC_value[9:0]+2],
Instrution_Memory[PC_value[9:0]+3]};

endmodule
```

---

2. 一句话总结这个模块

IFU = 取指单元 = 流水线的起点

功能：管理 PC，计算下一条指令地址，从指令内存中取出指令！

---

3. 逐段超通俗解释

① 端口

```verilog
input nPC_sel, zero, clk, rst;
input j_sel;
output[31:0] instruction;
```

- `clk`：时钟
- `rst`：复位
- `nPC_sel`：分支信号（beq 用）

- `zero`：ALU 零标志
- `j_sel`：跳转信号（j 用）
- `instruction`：**取出来的指令**

---

② 指令存储器 + PC

```verilog
reg[7:0] Instrution_Memory[1023:0]; // 指令内存
reg[31:0] PC_value;                 // 程序计数器
```

- `Instrution_Memory`：存放所有指令
- `PC_value`：**当前指令地址**

---

③ 复位 & PC 更新（最核心）

```verilog
always@(posedge clk, posedge rst)
begin
    if(rst)
        PC_value <= 32'h0000_3000;  // 复位地址
    else if(j_sel==1'b1)
        PC_value <= PC_ext;         // 跳转
    else
        PC_value <= PC_new;         // 正常 +4
end
```

意思：

- 复位 → PC 从 0x3000 开始
- 跳转指令 → PC 跳转到目标地址
4. 否则 → PC = PC + 4

---

④ 计算下一条指令地址

```verilog
assign ins0 = PC_value + 4;            // 正常：+4
assign ins1 = ins0 + PC_ext;           // 分支跳转
assign PC_new = (nPC_sel && zero) ? ins1 : ins0;
```

意思：

5. **正常执行**：PC = PC + 4
- **beq 分支成立**：PC 跳转到目标地址

---

⑤ 取出指令（最终目的）

```verilog
assign instruction = {
    Instrution_Memory[PC_value[9:0]],
    Instrution_Memory[PC_value[9:0]+1],
    Instrution_Memory[PC_value[9:0]+2],
    Instrution_Memory[PC_value[9:0]+3]
};
```

意思：

根据 PC 地址，从指令内存里取出 32 位指令
送给后面的流水线！

ALU.v

完整代码回顾

```verilog
// ALU 模块定义：模块名 ALU，端口列表 data1, data2, zero, Addr, ALUctr,
```

```
out
module ALU(data1, data2, zero, Addr, ALUctr, out);
input[31:0] data1, data2;  // 两个 32 位输入操作数
input[1:0] ALUctr;         // 2 位 ALU 控制信号（选择运算类型）

output[31:0] Addr;         // 32 位地址输出（给分支/访存用）
output reg[31:0] out;      // 32 位 ALU 运算结果输出（寄存器型）
output zero;               // 零标志位（判断结果是否为 0）

// 组合逻辑：输入变化时立即执行，无时钟延迟
always@(*)
begin
    // 根据 ALU 控制信号选择运算
    case(ALUctr)
        2'b00: begin out <= data1 + data2; end  // 加法
        2'b01: begin out <= data1 - data2; end  // 减法
        2'b10: begin out <= data1 | data2; end  // 按位或
    endcase
end

// 组合逻辑赋值：结果为 0 时 zero=1，否则 zero=0
assign zero = (out==32'b0)?1:0;
// 直接将 ALU 结果赋值给地址输出端口
assign Addr = out;

endmodule
```

DM.v

---

1. 完整 DM 数据存储器模块代码

```verilog
module DM(data_in, clk, WrEn, Addr, out, rst);
```

```
input[31:0] data_in, Addr;
input WrEn, clk, rst;
output[31:0] out;

reg[7:0] memory[1023:0];

always@(negedge clk)
begin
if(WrEn == 1'b1)
begin
memory[Addr[9:0]+3] <= data_in[31:24];
memory[Addr[9:0]+2] <= data_in[23:16];
memory[Addr[9:0]+1] <= data_in[15:8];
memory[Addr[9:0]] <= data_in[7:0];
end
end

assign  out = {memory[Addr[9:0]+3], memory[Addr[9:0]+2],
memory[Addr[9:0]+1], memory[Addr[9:0]]};

endmodule
```

第 8–16 行：写内存逻辑（核心）

```verilog
always@(negedge clk)
begin
if(WrEn == 1'b1)
begin
memory[Addr[9:0]+3] <= data_in[31:24];
memory[Addr[9:0]+2] <= data_in[23:16];
memory[Addr[9:0]+1] <= data_in[15:8];
memory[Addr[9:0]] <= data_in[7:0];
end
end
```

- `always@(negedge clk)`：**时钟下降沿触发写入**
- `WrEn == 1'b1`：**写使能有效时才写**
- 把 32 位数据拆成 4 个字节，写入连续 4 个地址：
  - `data_in[31:24]` → 地址 +3
  - `data_in[23:16]` → 地址 +2
  - `data_in[15:8]` → 地址 +1
  - `data_in[7:0]` → 基地址
- 这种存储方式叫 **大端模式（Big-Endian）**

第 18 行：读内存逻辑

```verilog
assign out = {memory[Addr[9:0]+3], memory[Addr[9:0]+2],
memory[Addr[9:0]+1], memory[Addr[9:0]]};
```

- **组合逻辑读，不需要时钟**
- 从连续 4 个字节中，把数据拼回 32 位
- 地址一旦变化，立即输出数据

ID

GPR.v

1. 完整代码

```verilog
module GPR(
    rw, ra, rb, regWr, busW, clk, busA, busB, data_in, rst
);
input[4:0] rw, ra, rb;
input[31:0] busW;
```

```
input clk, regWr, rst;
output[31:0] busA, busB, data_in;

// 32 个 32 位通用寄存器
reg[31:0] registers[31:0];

integer i;
// 复位：全部清零
always@(posedge rst)
begin
    if(rst)
        for(i=0; i<32; i = i + 1)
        begin
            registers[i] <= 0;
        end
end

// 时钟上升沿写寄存器
always@(posedge clk)
begin
    if(regWr)
        registers[rw] <= busW;

    // 0 号寄存器永远是 0
    registers[0] <= 0;
end

// 组合逻辑读寄存器
assign busA = registers[ra];
assign busB = registers[rb];
assign data_in = registers[rb];

endmodule
```

2. 逐行逐段超通俗解释

🔥 一句话核心总结

GPR = General Purpose Register = 通用寄存器堆
作用：CPU 用来临时存放数据的 32 个小抽屉，可读可写。

---

1. 端口是什么意思？

```verilog
input[4:0] rw, ra, rb;
input[31:0] busW;
input clk, regWr, rst;
output[31:0] busA, busB, data_in;
```

读端口（2 个读口）

- **ra**：读 A 端口的地址（读第几个寄存器）
- **rb**：读 B 端口的地址
- **busA**：从 ra 读出的数据
- **busB**：从 rb 读出的数据

写端口（1 个写口）

- **rw**：要写入的寄存器地址
- **busW**：要写入的数据
- **regWr**：写使能（=1 才能写）

其他

- `clk`：时钟
- `rst`：复位
- `data_in`：等于 busB，给后面模块用

---

2. 寄存器堆定义

```verilog
reg[31:0] registers[31:0];
```

5. **32 个寄存器**
6. 每个 **32 位**
- 名字：`$0, $1, $2 ... $31`

---

3. 复位功能

```verilog
always@(posedge rst)
begin
    if(rst)
        for(i=0; i<32; i = i + 1)
            registers[i] <= 0;
end
```

- 复位时 **32 个寄存器全部清 0**
- 保证 CPU 从头开始

---

4. 写寄存器（最核心）

```verilog
always@(posedge clk)
begin
    if(regWr)
        registers[rw] <= busW;

    registers[0] <= 0;
end
```

功能：

- **regWr = 1 时**

在时钟上升沿，把 `busW` 数据写入 `rw` 号寄存器

- **$0 永远等于 0**

硬件强制清零，不能修改

EXT.v

完整版

```verilog
// 模块名：EXT = 扩展器（把 16 位立即数 变成 32 位）
module 扩展器(
    16 位立即数， 32 位输出， 扩展方式选择
);

// 输入：16 位的立即数
input[15:0] 16 位立即数;

// 输入：扩展方式选择（00、01、10 三种）
input[1:0] 扩展方式选择;

// 输出：扩展后的 32 位结果
output reg[31:0] 32 位输出;

// 组合逻辑：只要输入变，输出立刻跟着变
always@(*)
begin
    // 如果扩展方式 = 00：零扩展（高位补 0）
    if(扩展方式选择 == 2'b00)
    begin
        32 位输出 <= { 16 位 0, 16 位立即数 };
    end
```

```
    // 如果扩展方式 = 01：符号扩展（最高位复制 16 次）
    else if(扩展方式选择 == 2'b01)
    begin
        32 位输出 <= {{16{16 位立即数[15]}}, 16 位立即数};
    end

    // 如果扩展方式 = 10：高位左移（后面补 0）
    else if(扩展方式选择 == 2'b10)
    begin
        32 位输出 <= {16 位立即数, 16 位 0};
    end
end

endmodule
```

逐行超简单解释

1. 模块定义 + 端口

```verilog
module EXT(imm16, out32, ExtOp);
input[15:0] imm16;    // 输入：一个 16 位的数
input[1:0] ExtOp;     // 输入：选择怎么扩展（3 种方式）
output reg[31:0] out32; // 输出：扩展后的 32 位数
```

2. 组合逻辑（只要输入变，输出立刻变）

```verilog
always@(*)
```

意思：**不用等时钟，马上算、马上出结果！**

---

3. 三种扩展方式（最核心）

① ExtOp = 00 → 零扩展

```verilog
out32 <= {16'b0, imm16};
```

前面补 16 个 0

例子：

16 位：1234

变成 32 位：0000 1234

---

② ExtOp = 01 → 符号扩展（最常用）

```verilog
out32 <= {{16{imm16[15]}}, imm16};
```

看最高位是 0 还是 1，复制 16 次补在前面

- 最高位 = 0 → 前面补 0
- 最高位 = 1 → 前面补 1

目的：**让负数也能正确表示**

---

③ ExtOp = 10 → 左移 16 位（后面补 0）

```verilog
out32 <= {imm16, 16'b0};
```

把 16 位数放左边，右边补 16 个 0

例子：

16 位：1234

变成 32 位：1234 0000

controller.v

一、完整代码

```verilog
module controller(
    instruction, RegDst, RegWr, nPC_sel, ExtOp,
    ALUSrc, ALUctr, MemWr, MemtoReg, j_sel
);

input[31:0] instruction;       // 输入：32 位指令
output[1:0] ExtOp, ALUctr;     // 输出：2 位控制信号
output RegDst, RegWr, ALUSrc, MemWr, MemtoReg, j_sel,nPC_sel;

wire[5:0] type = instruction[31:26];  // 指令高 6 位：操作码
wire[5:0] LSBs = instruction[5:0];    // 指令低 6 位：功能码

// 识别每条指令
assign addu = (type == 6'b0) && (LSBs == 6'b100001);
assign subu = (type == 6'b0) && (LSBs == 6'b100011);
assign ori  = (type == 6'b001101);
assign lw   = (type == 6'b100011);
assign sw   = (type == 6'b101011);
assign beq  = (type == 6'b000100);
assign lui  = (type == 6'b001111);
assign j    = (type == 6'b000010);

// 产生所有控制信号
assign RegDst    = addu | subu;
assign ALUSrc    = ori | lw | sw | lui;
assign MemtoReg  = lw;
assign RegWr     = addu | subu | ori | lw | lui;
assign MemWr     = sw;
assign j_sel     = j;
assign nPC_sel   = beq;
```

```
assign ExtOp     = {lui, lw|sw|j};
assign ALUctr    = {ori|lui,subu|beq|j};

endmodule
```

二、解释

1. 这个模块是干嘛的？

控制器 = CPU 的大脑 / 指挥官

- 输入：**一条指令**
- 输出：**一堆控制命令**
- 功能：**看懂指令 → 指挥所有硬件干活**

---

2. 输入 / 输出是什么？

输入

- **instruction**：32 位指令（从指令内存取出来的）

输出（全部是“命令”）

- RegDst：选择写哪个寄存器
- RegWr：是否写寄存器
- nPC_sel：是否分支跳转
- ExtOp：立即数扩展方式
- ALUSrc：ALU 第二个操作数来源
- ALUctr：ALU 做什么运算

6. MemWr：是否写内存
7. MemtoReg：结果来自内存还是 ALU

- j_sel：是否无条件跳转

3. 拆指令

```verilog
wire[5:0] type = instruction[31:26];  // 高 6 位：指令类型
wire[5:0] LSBs = instruction[5:0];    // 低 6 位：具体功能
```

- 所有指令长 32 位
- 看**高 6 位**就能知道大概是什么指令
- R 型指令（加法 / 减法）还要看**低 6 位**

4. 识别指令（这一段就是“认指令”）

```verilog
assign addu = ...  // 加法
assign subu = ...  // 减法
assign ori  = ...  // 立即数或
assign lw   = ...  // 读内存
assign sw   = ...  // 写内存
assign beq  = ...  // 条件相等跳转
assign lui  = ...  // 加载高位立即数
assign j    = ...  // 无条件跳转
```

作用：看到二进制指令 → 知道它是哪条指令

5. 产生控制信号（核心！大脑发命令）

```verilog
assign RegDst    = addu | subu;      // 加法减法：写 rd 寄存器
assign ALUSrc    = ori | lw | sw...; // 用立即数做 ALU 输入
assign MemtoReg  = lw;               // lw 指令：数据来自内存
assign RegWr     = 加法、减法、ori...;// 这些指令要写回寄存器
```

```
assign MemWr     = sw;              // sw 指令：写内存
assign j_sel     = j;               // j 指令：跳转
assign nPC_sel   = beq;             // beq 指令：分支
assign ExtOp     = {lui, lw|sw|j};   // 扩展方式
assign ALUctr    = {ori|lui,subu|beq};// ALU 运算类型
```